

UNIVERSIDAD DE SALAMANCA

GRADO EN INGENIERIA INFORMATICA

DocumentChain

Sistema de Gestion Documental mediante Blockchain



David Pérez Velasco

Tutores:

Gabriel Villarrubia González

Sergio García González

Julio 2026

Trabajo de Fin de Grado

Consentimiento informado del tutor

El Dr. Gabriel Villarrubia González, profesor del Departamento de Informática y Automática de la Universidad de Salamanca, y el Dr. Sergio García González, certifican que el trabajo titulado “DocumentChain: Sistema de Gestión Documental mediante Blockchain” ha sido realizado por D. David Pérez Velasco, y constituye el trabajo realizado de cara a la superación de la asignatura Trabajo de Fin de Grado de la titulación Grado Universitario en Ingeniería Informática de la Universidad de Salamanca.

En Salamanca, a _____

Fdo.: Gabriel Villarrubia González

Fdo.: Sergio García González

Resumen

Los sistemas de gestion documental convencionales ofrecen control de versiones y comparticion, pero carecen de mecanismos descentralizados que garanticen la inmutabilidad del contenido y la verificacion criptografica de las operaciones, por lo que la autenticidad y la trazabilidad siguen siendo problemas sin resolver.

Ante esta problematica surge **DocumentChain**, un sistema de gestion documental que integra tecnologia blockchain para garantizar la autenticidad, trazabilidad e inmutabilidad de los documentos gestionados. El proyecto combina una aplicacion web desarrollada con React y TypeScript, una API REST con Node.js y Express, y una capa de almacenamiento distribuido mediante IPFS. La blockchain local (Hardhat) registra los hashes de contenido y las firmas digitales, proporcionando un historial inmutable de operaciones.

El sistema implementa un esquema de seguridad por capas (autenticacion JWT con verificacion de email, cifrado AES-256-GCM con intercambio ECDH y registro blockchain verificable), soporta la firma digital mediante wallets compatibles con EIP-6963 y ofrece un mecanismo de verificacion publica accesible sin autenticacion. El resultado es una plataforma funcional que demuestra la viabilidad tecnica de combinar gestion documental con trazabilidad descentralizada, sentando las bases para futuras mejoras y ampliaciones.

Palabras clave: gestion documental, blockchain, autenticidad, firma digital, IPFS, cifrado, trazabilidad, descentralizacion.

Abstract

In the field of digital document management, authenticity and traceability represent critical challenges that traditional solutions fail to address satisfactorily. Conventional cloud storage systems offer versioning and sharing capabilities, but lack decentralized mechanisms that guarantee content immutability and cryptographic verification of performed operations.

In response to this problem, **DocumentChain** emerges as a document management system that integrates blockchain technology to ensure the authenticity, traceability and immutability of managed documents. The project combines a web application developed with React and TypeScript, a REST API with Node.js and Express, and a distributed storage layer through IPFS. The local blockchain (Hardhat) registers content hashes and digital signatures, providing an immutable history of operations.

The system implements a layered security scheme (JWT-based authentication with email verification, AES-256-GCM encryption with ECDH key exchange, and verifiable blockchain registry), supports digital signing through EIP-6963 compatible wallets, and offers a public verification mechanism accessible without authentication. The result is a functional platform that demonstrates the technical feasibility of combining document management with decentralized traceability, laying the groundwork for future improvements and extensions.

Keywords: document management, blockchain, authenticity, digital signature, IPFS, encryption, traceability, decentralization.

Índice

Resumen	2
Abstract	2
Glosario	7
1. Introduccion	8
1.1. Contexto y motivacion	8
1.2. Problematica actual	8
1.3. Solucion propuesta	8
1.4. Estructura del documento	8
1.5. Participantes del proyecto	9
2. Estudio de mercado	9
2.1. Aplicaciones comerciales existentes	9
2.2. Deficiencias identificadas	10
2.3. Propuesta diferenciadora	10
2.4. Investigacion academica relacionada	11
3. Objetivos	11
3.1. Objetivos del sistema	11
3.2. Objetivos personales	12
4. Conceptos teoricos	12
4.1. Problemas de autenticidad documental	12
4.2. Marco legal europeo	12
4.3. Impacto economico y social de la falsificacion documental	12
4.4. La necesidad de descentralizacion	13
4.5. Tecnologias habilitadoras	13
5. Herramientas y tecnicas empleadas	13
5.1. Herramientas de servidores	13
5.2. Herramientas de blockchain y almacenamiento	13
5.3. Herramientas de control de versiones	14
5.4. Herramientas de documentacion	14
5.5. Herramientas CASE y de estimacion	14
5.6. Inteligencia Artificial	14
6. Aspectos relevantes del proyecto	14
6.1. Marco de trabajo	14
6.2. Estimacion temporal del proyecto	15
6.3. Planificacion temporal	15
6.4. Especificacion de requisitos	15
6.5. Analisis de requisitos	17
6.6. Diseno del sistema	20
6.7. Implementacion	23
6.8. Funcionalidad del sistema	23
6.8.1. Gestion documental	24
6.8.2. Firma digital y comparticion	25
6.8.3. Verificacion publica y administracion	27

7. Limitaciones del sistema	29
7.1. Restricciones tecnologicas y de infraestructura	30
7.2. Restricciones de seguridad y auditoria	30
7.3. Acceso a usuarios y pruebas de usabilidad	30
7.4. Restricciones regulatorias y de compliance	31
7.5. Restricciones del proyecto	31
8. Conclusiones	31
8.1. Validacion de objetivos	31
9. Lineas de trabajo futuras	32
10. Referencias y bibliografia	33

Índice de figuras

1.	Arquitectura del sistema DocumentChain	18
2.	Diagrama de secuencia de analisis BCE: UC-0015 Subir Documento	19
3.	Diagrama de secuencia de diseno: UC-0015 Subir Documento	20
4.	Modelo de despliegue con contenedores Docker	21
5.	Diagrama entidad-relacion de la base de datos	21
6.	Patron prepare/confirm para transacciones blockchain	22
7.	Flujo de sincronizacion bidireccional mediante Event Listener	22
8.	Vista principal de gestion documental	24
9.	Modal de subida de documentos con metadatos	25
10.	Modal de firma digital con wallet blockchain	26
11.	Modal de comparticion de documentos con permisos	27
12.	Pagina de verificacion publica de autenticidad de documentos	28
13.	Panel de administracion del sistema	29

Índice de cuadros

1.	Deficiencias identificadas en las soluciones comerciales actuales	10
2.	Pilares diferenciadores de DocumentChain	11
3.	Fases del Proceso Unificado aplicadas al proyecto	15
4.	Ejemplo de especificacion de caso de uso: UC-0015 Subir Documento	16
5.	Requisitos no funcionales del sistema	17
6.	Registro de riesgos del proyecto	23
7.	Funcionalidades diferidas por restricciones temporales	30
8.	Validacion de objetivos funcionales	32

Glosario

API (Application Programming Interface)

Conjunto de definiciones y protocolos que permiten la comunicacion entre diferentes componentes software.

Blockchain

Estructura de datos distribuida e inmutable compuesta por bloques enlazados mediante funciones hash.

CID (Content Identifier)

Identificador unico de contenido en IPFS basado en hashing criptografico.

ECDH (Elliptic Curve Diffie-Hellman)

Protocolo de intercambio de claves basado en curvas elipticas.

EIP-6963

Propuesta de mejora de Ethereum para la deteccion de wallets mediante eventos de ventana.

personal_sign

Metodo de firma de mensajes arbitrarios mediante una wallet Ethereum, empleado para acreditar la posesion de la direccion que firma una operacion.

IPFS (InterPlanetary File System)

Protocolo descentralizado de almacenamiento y distribucion de archivos.

JWT (JSON Web Token)

Token firmado digitalmente utilizado para la autenticacion entre partes.

SPA (Single Page Application)

Aplicacion web que carga una unica pagina HTML y actualiza el contenido dinamicamente.

TOTP (Time-based One-Time Password)

Contrasena de un solo uso basada en tiempo para autenticacion multifactor (no implementada en la version actual del sistema).

UCP (Use Case Points)

Tecnica de estimacion del tamano y esfuerzo de proyectos software basada en casos de uso.

EVM (Ethereum Virtual Machine)

Maquina virtual que ejecuta smart contracts en la red Ethereum.

1. Introduccion

1.1. Contexto y motivacion

La digitalizacion ha cambiado por completo la forma en que gestionamos la informacion. Herramientas como Google Drive o Dropbox permiten compartir archivos y mantener un historial de versiones, pero tienen un problema de fondo: todo depende de un unico proveedor. Si ese proveedor modifica un archivo, lo censura o simplemente cierra el servicio, el usuario se queda sin garantias.

La tecnologia blockchain ofrece una alternativa interesante a este modelo. Al mantener un registro descentralizado que nadie puede modificar unilateralmente, permite guardar huellas digitales de los documentos de forma que cualquiera pueda verificar su autenticidad sin necesidad de confiar en un tercero. Esto fue lo que motivo este trabajo: comprobar si era viable construir una plataforma de gestion documental donde la prueba de autenticidad no dependa de la palabra de una empresa, sino de las matematicas.

1.2. Problematica actual

El problema de la autenticidad documental va mas alla de lo tecnologico. Hoy en dia modificar un PDF o un titulo academico es sorprendentemente facil, y las victimas de estas falsificaciones a menudo no tienen forma de demostrarlo. Los sistemas de firma digital tradicionales existen, pero su adopcion es baja porque requieren certificados que son complicados de gestionar para el usuario medio.

A esto se le suma que los sistemas de almacenamiento actuales son cajas negras: el proveedor controla los registros de auditoria y el usuario no puede comprobar por si mismo si un documento ha sido alterado. Las filtraciones masivas de datos de los ultimos anos demuestran que confiar toda la informacion a un unico servidor no es la mejor estrategia.

1.3. Solucion propuesta

DocumentChain es una plataforma web que combina tres tecnologias para resolver estos problemas. Por un lado, usa blockchain para registrar huellas inmutables de cada documento. Por otro, emplea IPFS como almacenamiento descentralizado, donde los archivos se identifican por su contenido y no por el servidor que los aloja. Y por ultimo, cifra los documentos privados de extremo a extremo para que ni siquiera el servidor pueda leer su contenido.

La aplicacion se ha construido con React y TypeScript en el frontend, Node.js y Express en el backend, PostgreSQL para los datos operativos y contratos inteligentes en Solidity para el registro en blockchain. El usuario firma las operaciones con su wallet (compatible con EIP-6963) y el sistema permite que cualquier persona, sin necesidad de crear una cuenta, verifique si un documento es autentico consultando directamente la blockchain.

1.4. Estructura del documento

El presente documento se organiza en diez capitulos. El Capitulo 1 introduce el contexto, la problematica, la solucion propuesta y la estructura del documento. El Capitulo 2 recoge un estudio de mercado con el analisis de soluciones comerciales, sus deficiencias, la propuesta diferenciadora de DocumentChain y la investigacion academica de referencia. Los objetivos funcionales y personales del proyecto se detallan en el Capitulo 3. El Capitulo 4 expone los conceptos teoricos: autenticidad documental, marco legal, impacto economico, necesidad de descentralizacion y tecnologias habilitadoras. A continuacion, el Capitulo 5 describe las herramientas de desarrollo, los estandares empleados y el uso de inteligencia artificial. El Capitulo 6 aborda los aspectos mas relevantes del desarrollo, desde el marco de trabajo y la estimacion hasta la planificacion,

los requisitos, el analisis, el diseno, la implementacion y la funcionalidad resultante. Las limitaciones de tiempo, tecnologicas, de seguridad y de acceso a usuarios se tratan en el Capitulo 7. El Capitulo 8 presenta las conclusiones del proyecto, con la validacion de los objetivos alcanzados y una reflexion sobre el trabajo realizado. Las lineas de trabajo futuras y las mejoras propuestas para la evolucion del sistema figuran en el Capitulo 9. Por ultimo, el Capitulo 10 recopila las fuentes consultadas en formato estandarizado.

Para completar la explicacion de esta memoria, se acompaña de nueve anexos donde se recoge la documentacion tecnica exhaustiva. El Anexo I contiene el catalogo completo de requisitos funcionales, no funcionales y de informacion, junto con los actores, los diagramas de casos de uso y la matriz de rastreabilidad. El Anexo II detalla la metodologia de estimacion, el calculo de Use Case Points, la planificacion temporal y los diagramas de Gantt. El Anexo III documenta el analisis y diseno del sistema: modelo de dominio, diagramas BCE, arquitectura, clases de diseno, base de datos y modelo de despliegue. La documentacion tecnica del codigo, las tecnologias, los modulos, los smart contracts y la arquitectura de cifrado se recogen en el Anexo IV. El Anexo V presenta el plan de seguridad con los activos criticos, las amenazas, la matriz de riesgos y las medidas implementadas. El Anexo VI es el manual de usuario, con la guia de instalacion, configuracion y uso paso a paso de la aplicacion. El Anexo VII constituye el manual de montaje y despliegue, con los requisitos, la instalacion mediante Docker Compose y la verificacion del despliegue. La declaracion de las herramientas de inteligencia artificial empleadas y la transparencia en su uso se documentan en el Anexo VIII. Por ultimo, el Anexo IX expone la metodologia de Diseno Centrado en el Usuario aplicada para asegurar la usabilidad del sistema.

1.5. Participantes del proyecto

El presente Trabajo de Fin de Grado ha sido desarrollado en el marco de la asignatura Trabajo de Fin de Grado del Grado en Ingenieria Informatica de la Universidad de Salamanca durante el curso academico 2025-2026. Los participantes en el proyecto son:

- **Autor:** D. David Perez Velasco, alumno del Grado en Ingenieria Informatica de la Universidad de Salamanca, responsable del analisis, diseno, implementacion, documentacion y despliegue del sistema DocumentChain.
- **Tutor academico:** Dr. D. Gabriel Villarrubia Gonzalez, profesor del Departamento de Informatica y Automatica de la Universidad de Salamanca, responsable de la supervision tecnica y metodologica del proyecto.
- **Cotutor:** Dr. D. Sergio Garcia Gonzalez, responsable de la supervision complementaria del desarrollo y la documentacion del sistema.

2. Estudio de mercado

En este apartado se ha realizado un estudio de las soluciones comerciales existentes con el fin de identificar sus carencias y establecer una propuesta diferenciadora para DocumentChain.

2.1. Aplicaciones comerciales existentes

Antes de comenzar el desarrollo, se llevo a cabo un estudio del mercado disponible tratando de encontrar productos similares u orientados al mismo proposito. Se analizaron cuatro productos comerciales y una linea de investigacion academica.

DocuSign es el actor dominante en firma electronica, operando bajo un modelo SaaS centralizado. Ofrece flujos de trabajo configurables y cumplimiento con eIDAS, pero su funcion

principal gira en torno a la firma, no al ciclo de vida documental completo. La arquitectura centralizada implica que DocuSign actúa como único guardian de la verdad, por lo que no permite la verificación independiente por terceros.

Adobe Sign se integra con Adobe Document Cloud y soporta firmas cualificadas. Comparte las limitaciones de DocuSign: centralización, ausencia de blockchain y dependencia del proveedor, por lo que presenta los mismos problemas de confianza que el anterior.

Google Drive y Dropbox ofrecen almacenamiento, sincronización y control de versiones básico. Sin embargo, su propósito es la productividad, no la garantía de autenticidad, por lo que los registros pueden ser modificados por el administrador sin dejar rastro.

OriginStamp adopta un enfoque de *proof of existence* registrando hashes en la blockchain de Bitcoin. No almacena documentos ni gestiona permisos, por lo que resulta insuficiente para una gestión documental completa.

2.2. Deficiencias identificadas

El análisis de las soluciones existentes revela deficiencias en cinco dimensiones: arquitectura de confianza centralizada con punto único de fallo, ausencia de cifrado de extremo a extremo, registros de auditoría mutables, fragmentación funcional que obliga a combinar múltiples herramientas y modelos de precios que generan barreras para pequeñas organizaciones. La Tabla 1 resume estas limitaciones.

Dimension	Deficiencia	Impacto	Soluciones afectadas
Arquitectura de confianza	Dependencia de autoridad central única	Riesgo de censura, suspensión y punto único de fallo	DocuSign, Adobe Sign, Drive, Dropbox
Seguridad del contenido	Ausencia de cifrado de extremo a extremo	El proveedor puede acceder al contenido	Todas las centralizadas
Trazabilidad histórica	Registros de auditoría mutables	Imposibilidad de demostrar la secuencia real de operaciones	Todas excepto OriginStamp
Funcionalidad documental	Fragmentación entre firma, almacenamiento y prueba	Necesidad de múltiples herramientas	Todas
Coste y accesibilidad	Suscripción por usuario o transacción	Barrera para PYMEs y usuarios individuales	DocuSign, Adobe Sign

Cuadro 1: Deficiencias identificadas en las soluciones comerciales actuales

2.3. Propuesta diferenciadora

DocumentChain integra en una única plataforma gestión documental, cifrado, almacenamiento distribuido y registro blockchain. A diferencia de las soluciones analizadas, no depende de un proveedor centralizado ni obliga a combinar varias herramientas, por lo que ofrece una experiencia más unificada. La Tabla 2 resume sus pilares diferenciadores.

Pilar diferenciador	Descripcion tecnica	Ventaja competitiva
Gestion documental completa	CRUD, carpetas, versiones, comparticion, transferencia	Ciclo de vida integral sin fragmentacion
Cifrado de extremo a extremo	AES-256-GCM con ECDH P-256 en el cliente	Confidencialidad incluso ante compromiso del servidor
Almacenamiento distribuido	IPFS con CID	Resistencia a la censura y punto unico de fallo
Registro inmutable	Smart contracts con hashes SHA-256 y firmas mediante wallet	Trazabilidad verificable sin intermediarios
Verificacion publica	Endpoint abierto por hash o txHash	Transparencia universal sin registro previo

Cuadro 2: Pilares diferenciadores de DocumentChain

2.4. Investigacion academica relacionada

La gestion documental descentralizada ha recibido creciente atencion academica. Zyskind, Nathan y Pentland (2015) proponen un sistema donde la blockchain actua como registro de control de acceso y los datos se almacenan off-chain, modelo adoptado por DocumentChain. Liang et al. (2017) presentan en ProvChain una arquitectura de trazabilidad de datos en la nube con registro blockchain de cada operacion. En cuanto al almacenamiento distribuido, Benet (2014) establece el protocolo IPFS que DocumentChain utiliza como capa de almacenamiento. La contribucion de este proyecto radica en integrar todas estas capacidades en un unico sistema documentado, funcional y reproducible.

3. Objetivos

En este apartado se recogen los objetivos que el sistema debe cumplir para considerarse totalmente funcional, asi como los objetivos personales que se pretenden alcanzar con el desarrollo de este proyecto.

3.1. Objetivos del sistema

El objetivo principal del proyecto es disenar e implementar un sistema de gestion documental con trazabilidad blockchain que garantice la autenticidad, integridad y confidencialidad de los documentos mediante cifrado de extremo a extremo y registro inmutable. En particular, se definen los siguientes objetivos:

- **Gestion documental completa:** permitir el ciclo de vida completo de los documentos digitales (crear, versionar, compartir, firmar, archivar y eliminar).
- **Registro inmutable en blockchain:** integrar una capa blockchain para el registro de hashes de documentos y firmas digitales, garantizando trazabilidad verificable.
- Que el sistema permita firmar documentos mediante wallets compatibles con EIP-6963, generando mensajes de firma mediante wallet y registrando la firma en blockchain.
- Se necesita un modulo de cifrado AES-256-GCM en el cliente, con intercambio de claves ECDH P-256, para que el contenido de los documentos privados resulte inaccesible incluso para el servidor.

- **Verificación pública:** proporcionar un mecanismo de verificación de autenticidad accesible sin necesidad de registro previo.
- **Panel de administración:** diseñar un panel para la gestión de usuarios, roles y supervisión del sistema.
- **Documentación exhaustiva:** documentar el sistema siguiendo la normativa CCII-N2016-02 de la Universidad de Salamanca.

3.2. Objetivos personales

- **Profundizar en tecnologías blockchain:** adquirir competencia práctica en el desarrollo de smart contracts y su integración en aplicaciones web.
- **Desarrollo full-stack:** dominar un stack tecnológico moderno que abarca React, Node.js, Solidity, Docker e IPFS de forma coordinada.
- **Ingeniería del software:** aplicar metodologías de análisis, diseño y estimación basadas en el Proceso Unificado y Use Case Points.
- **Gestión integral de proyectos:** demostrar la capacidad de conducir un proyecto complejo desde la concepción hasta la documentación final.

4. Conceptos teóricos

En este apartado se abordan conceptos teóricos necesarios para el correcto entendimiento del proyecto, facilitando así la lectura y comprensión por parte del lector.

4.1. Problemas de autenticidad documental

La autenticidad de los documentos es un problema real y cotidiano. Modificar un PDF, un título universitario o un informe médico es sorprendentemente fácil con las herramientas actuales, y las víctimas rara vez pueden demostrarlo. Los sistemas de firma digital basados en PKI llevan años intentando resolver esto, pero su complejidad (gestión de certificados, autoridades centralizadas) hace que pocas organizaciones pequeñas los usen realmente.

4.2. Marco legal europeo

El sistema se ha diseñado teniendo en cuenta dos normativas europeas. El reglamento eIDAS (UE 910/2014) establece tres niveles de firma electrónica y da validez legal a las firmas digitales. El RGPD (UE 2016/679) obliga a proteger los datos personales. DocumentChain intenta cumplir ambos: los documentos privados viajan cifrados, en la blockchain solo se guardan hashes (no el contenido), y la recuperación de archivos se hace por CID, sin depender de un proveedor concreto.

4.3. Impacto económico y social de la falsificación documental

Las cifras son llamativas. Según la Association of Certified Fraud Examiners, las empresas pierden de media un 5 % de sus ingresos anuales por fraude documental. El mercado de títulos falsos mueve entre 1.500 y 2.000 millones de dólares al año. Detrás de cada número hay personas que pierden oportunidades laborales o médicas por documentos que no son auténticos. La combinación de cifrado, blockchain e IPFS que se plantea aquí aborda directamente este problema.

4.4. La necesidad de descentralizacion

En un sistema tradicional, el administrador del servidor puede modificar los registros de auditoria sin que nadie se de cuenta. La blockchain cambia las reglas: una vez que un dato se escribe, modificarlo requeriria rehacer toda la cadena de bloques posterior, algo computacionalmente inviable. Esto sustituye la confianza en una persona por la confianza en las matematicas.

4.5. Tecnologias habilitadoras

Tres tecnologias sostienen el sistema. La blockchain de Ethereum guarda un registro inmutable de cada operacion sobre un documento (creacion, version, firma, comparticion). IPFS almacena los archivos identificandolos por su huella digital, no por el servidor donde estan, lo que permite verificar que un archivo no ha sido modificado simplemente comprobando su identificador. Y el cifrado AES-256-GCM protege el contenido de los documentos privados.

5. Herramientas y tecnicas empleadas

En este apartado se presentan las herramientas y tecnologias que se han utilizado durante el desarrollo del proyecto. Se ha dividido en varios bloques segun su funcion dentro del sistema.

5.1. Herramientas de servidores

TypeScript ha sido el lenguaje principal tanto en frontend como en backend. Su sistema de tipos permite detectar errores en tiempo de compilacion, lo cual ha resultado util en un proyecto con multiples capas donde los contratos de interfaz deben mantenerse coherentes entre modulos. **React 19** y **Vite 7** forman el frontend. React permite construir la interfaz con componentes reutilizables y Vite ofrece un entorno de desarrollo con recarga rapida.

Node.js y **Express 4** proporcionan el backend. La naturaleza asincrona de Node.js se adapta bien a las operaciones con IPFS y blockchain, mientras que Express facilita el enrutamiento y la gestion de middlewares.

PostgreSQL 16 y **Prisma** gestionan la persistencia. PostgreSQL se eligio por su soporte para transacciones ACID y Prisma genera automaticamente los tipos TypeScript desde el esquema de base de datos, por lo que se evitan errores de inconsistencia entre el modelo de datos y el codigo.

Para la gestion del estado en el frontend se han empleado **TanStack Query** para los datos del servidor y **Zustand** para el estado local. **Tailwind CSS** proporciona el diseno responsive. Todo el sistema se despliega con **Docker** y **Docker Compose**, lo que permite reproducir el entorno en cualquier maquina.

5.2. Herramientas de blockchain y almacenamiento

Solidity y **Hardhat** constituyen el entorno de desarrollo de los contratos inteligentes. Hardhat permite compilar y probar los contratos en una red local antes de cualquier despliegue, por lo que se reducen los errores en produccion.

ethers.js v6 es la biblioteca de interaccion con la blockchain, usada tanto en el frontend para firmar transacciones como en el backend para verificar eventos. **OpenZeppelin** proporciona patrones de seguridad ya auditados que se integran en los contratos.

IPFS gestiona el almacenamiento de archivos mediante un proveedor compatible con IPFS. En la configuracion local se utiliza Pinata por defecto, manteniendo la posibilidad de activar un nodo Kubo autoalojado si se configura el proveedor correspondiente. Al identificar los archivos por su hash en lugar de por su ubicacion, cualquier modificacion produce un identificador diferente, por lo que la integridad se verifica automaticamente.

5.3. Herramientas de control de versiones

- **Git**: sistema de control de versiones con flujo de trabajo basado en ramas feature/main.
- **GitHub**: plataforma de alojamiento del repositorio, con pull requests y CI/CD.

5.4. Herramientas de documentacion

- **PlantUML**: generacion de diagramas UML (casos de uso, clases, secuencia, componentes, despliegue, entidad-relacion).
- **Swagger/OpenAPI**: documentacion interactiva de la API REST.
- **NatSpec**: formato de comentarios para documentar contratos inteligentes en Solidity.
- **L^AT_EX**: composicion de la memoria y los anexos tecnicos.

5.5. Herramientas CASE y de estimacion

- **Visual Studio Code**: IDE principal con extensiones para TypeScript, React, Solidity y Prisma.
- **Postman**: herramienta para interaccion con la API REST durante el desarrollo.
- **EZEstimate**: herramienta para estimacion del esfuerzo mediante Use Case Points.
- **GanttProject**: planificacion temporal y elaboracion de diagramas de Gantt.

5.6. Inteligencia Artificial

Durante el desarrollo se ha hecho uso de asistentes de IA (ChatGPT, GitHub Copilot, Claude) exclusivamente para depuracion de errores, optimizacion de fragmentos de codigo, generacion de plantillas de documentacion y asesoramiento arquitectonico. Todo el contenido generado ha sido supervisado, verificado y adaptado manualmente. El Anexo VIII recoge el registro detallado de los prompts utilizados y la declaracion de autoria correspondiente.

6. Aspectos relevantes del proyecto

En este apartado se explican las partes mas importantes del desarrollo del proyecto, desde la fase inicial de estimacion hasta la fase final de implementacion.

6.1. Marco de trabajo

El marco de trabajo utilizado ha sido el **Proceso Unificado (UP)**, que define un desarrollo guiado por casos de uso, siguiendo un enfoque iterativo e incremental. Sus principales caracteristicas son: dirigido por casos de uso, centrado en la arquitectura, e iterativo e incremental. El UP se divide en cuatro fases: Inicio (definicion del alcance y requisitos), Elaboracion (arquitectura base y refinamiento), Construccion (implementacion con iteraciones) y Transicion (despliegue y documentacion).

La eleccion del UP frente a metodologias agiles puras se eligio UP porque para un TFG se necesita planificacion estructurada y trazabilidad clara entre requisitos, diseno e implementacion. El UP se adapto al contexto reduciendo la formalidad y acortando los ciclos de iteracion para ajustarse a la disponibilidad de un unico desarrollador.

Para obtener mas informacion sobre este apartado, consultar el Anexo II (Estimacion del tamano y esfuerzo).

6.2. Estimacion temporal del proyecto

Para comenzar con el proyecto, la primera tarea fue realizar una estimacion del tiempo de duracion mediante la tecnica de **Use Case Points (UCP)**, que permite cuantificar el tamano del proyecto en funcion de la complejidad de los actores y los casos de uso, ajustando el esfuerzo mediante factores tecnicos y ambientales. Para esta estimacion se utilizo la herramienta EZEstimate. Los resultados completos pueden consultarse en el Anexo II.

Para obtener mas informacion sobre este apartado, consultar el Anexo II.

6.3. Planificacion temporal

Una vez conocido el tiempo estimado, se procedio a realizar la planificacion temporal de las diferentes tareas siguiendo el esquema del Proceso Unificado. La Tabla 3 resume las fases, su duracion y los hitos principales alcanzados.

Fase	Periodo	Hitos principales
Inicio	Ene 2026 (2 sem)	Definicion de alcance, identificacion de actores y UC, seleccion del stack tecnologico
Elaboracion	Ene-Feb 2026 (4 sem)	Arquitectura de cifrado, diseno del smart contract, diagramas UML, redaccion de Anexos I y II
Construccion	Feb-May 2026 (14 sem)	Implementacion backend, frontend e integracion de subsistemas
Transicion	Jun 2026 (3 sem)	Dockerizacion, despliegue, documentacion tecnica, memoria y preparacion de defensa

Cuadro 3: Fases del Proceso Unificado aplicadas al proyecto

Para obtener mas informacion sobre este apartado, consultar el Anexo II (Planificacion temporal y diagramas de Gantt).

6.4. Especificacion de requisitos

Una vez finalizada la fase inicial de planificacion temporal, se paso a la fase de elicitation de requisitos software, donde se recogieron todos los requisitos que debe satisfacer el sistema. La especificacion detallada de cada caso de uso, incluyendo precondiciones, secuencia normal, postcondiciones y excepciones, se encuentra en el Anexo I.

A modo de ejemplo, la Tabla ?? muestra los objetivos funcionales del sistema, la Tabla ?? identifica los actores que interactuan con DocumentChain, y la Figura ?? presenta el diagrama general de casos de uso con la distribucion de los 43 casos de uso en siete paquetes funcionales (Tabla ??).

A titulo ilustrativo, la Tabla 4 muestra la especificacion completa de un caso de uso representativo del sistema: UC-0015 Subir Documento, que constituye la operacion principal del modulo de gestion documental e involucra las tres capas de persistencia (PostgreSQL, IPFS y blockchain). El Anexo I contiene las 43 especificaciones completas con este mismo nivel de detalle.

UC-0015: Subir Documento	
Descripcion	El usuario sube un archivo a la plataforma. El sistema cifra el contenido (si el documento es privado), lo almacena en IPFS y registra el hash en la blockchain para garantizar la inmutabilidad del registro.
Actores	Usuario registrado (ACT-01), Smart Contract (ACT-03), Nodo IPFS (ACT-04)
Precondicio- nes	El usuario ha iniciado sesion y tiene su cuenta verificada. Dispone de al menos una wallet conectada.
Secuencia normal	1. El usuario accede a la vista principal de documentos. 2. El usuario pulsa el boton "Subir documento". 3. El sistema muestra el modal de subida con campos de metadatos (nombre, descripcion, visibilidad, tags). 4. El usuario completa los metadatos y selecciona el archivo desde su dispositivo. 5. El sistema valida el tipo MIME y el tamano maximo del archivo. 6. Si el documento es privado, el sistema genera una clave simetrica AES-256-GCM, cifra el archivo y sube el resultado a IPFS. 7. Si el documento es publico, el archivo se sube a IPFS sin cifrar. 8. El sistema crea un registro en PostgreSQL con estado PREPARING y devuelve los datos de la transaccion al frontend. 9. El frontend solicita al usuario la firma de la transaccion mediante su wallet (MetaMask, WalletConnect). 10. El usuario revisa y firma la transaccion. 11. El frontend envia el transaction hash al backend para confirmacion. 12. El backend marca el registro como TX_SUBMITTED y el Event Listener actualiza el estado a SYNCED cuando la transaccion se confirma en blockchain. 13. El sistema muestra el documento en la lista principal con el estado "Sincronizado".
Postcondi- ciones	El documento queda registrado en la base de datos, su contenido cifrado almacenado en IPFS y su hash registrado en la blockchain. Es visible en la lista de documentos del usuario.
Excepciones	Archivo no soportado (400 Bad Request), tamano excede el limite (400), usuario sin wallet conectada (400), transaccion revertida en blockchain (estado FAILED), error de conexion con IPFS (503).

Cuadro 4: Ejemplo de especificacion de caso de uso: UC-0015 Subir Documento

Como complemento a los requisitos funcionales, la Tabla 5 sintetiza los requisitos no funcionales del sistema, organizados por categoria. La especificacion completa de todos los NFR se encuentra en el Anexo I.

ID	Categoria	Descripcion
NFR-0001	Seguridad	Cifrado AES-256-GCM para documentos privados con intercambio ECDH P-256. Autenticacion JWT con verificacion de email. Contraseñas derivadas con Argon2id.
NFR-0002	Rendimiento	Tiempo de respuesta maximo de 2 segundos para operaciones de lectura. Subida de archivos de hasta 50 MB. Paginacion en todos los listados.
NFR-0003	Usabilidad	Interfaz responsive adaptable a dispositivos moviles. Terminologia accesible sin conocimiento previo de blockchain. Indicadores de progreso en operaciones asincronas.
NFR-0004	Escalabilidad	Arquitectura contenerizada con Docker Compose. Servicios independientes escalables horizontalmente. Separacion de capas de persistencia.
NFR-0005	Disponibilidad	Sincronizacion periodica blockchain-base de datos. Recuperacion automatica ante reinicio del Event Listener. Tolerancia a fallos de red mediante reintentos.
NFR-0006	Mantenibilidad	Codigo documentado con JSDoc/NatSpec. API documentada con TypeDoc. Diagramas UML actualizados mediante PlantUML. Scripts de despliegue automatizados.

Cuadro 5: Requisitos no funcionales del sistema

Para obtener mas informacion sobre este apartado, consultar el Anexo I (Especificacion de requisitos).

6.5. Analisis de requisitos

Finalizada la fase de elicitation de requisitos, se paso a la fase de analisis, que se llevo a cabo siguiendo la metodologia BCE (Boundary-Control-Entity). Este patron organiza las clases del sistema en tres categorias: clases de frontera (Boundary), que gestionan la interaccion con los actores; clases de control (Control), que implementan la logica de negocio; y clases de entidad (Entity), que representan los objetos persistentes del dominio.

El analisis BCE se aplico a cada uno de los 43 casos de uso, generando los correspondientes diagramas de secuencia de analisis que muestran la interaccion entre las clases de frontera, control y entidad para cada flujo funcional. La Figura 1 (ver Seccion 6.6) muestra como esta organizacion se materializa en la arquitectura del sistema.

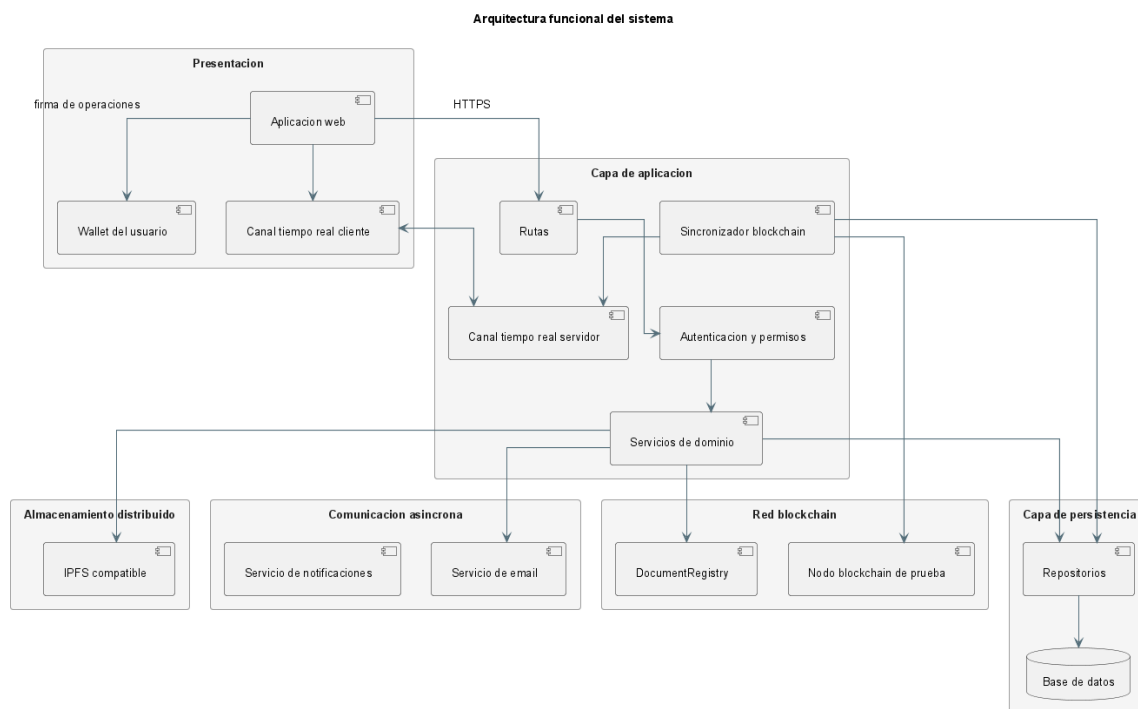


Figura 1: Arquitectura del sistema DocumentChain

A modo de ejemplo, la Figura 2 muestra el diagrama de secuencia de analisis BCE correspondiente al caso de uso UC-0015 (Subir Documento), que ilustra la aplicacion del patron Boundary-Control-Entity. La Figura 3 presenta el diagrama de secuencia de diseno para el mismo caso de uso, reflejando la implementacion real con React (frontend), Express (backend), IPFS y el smart contract.

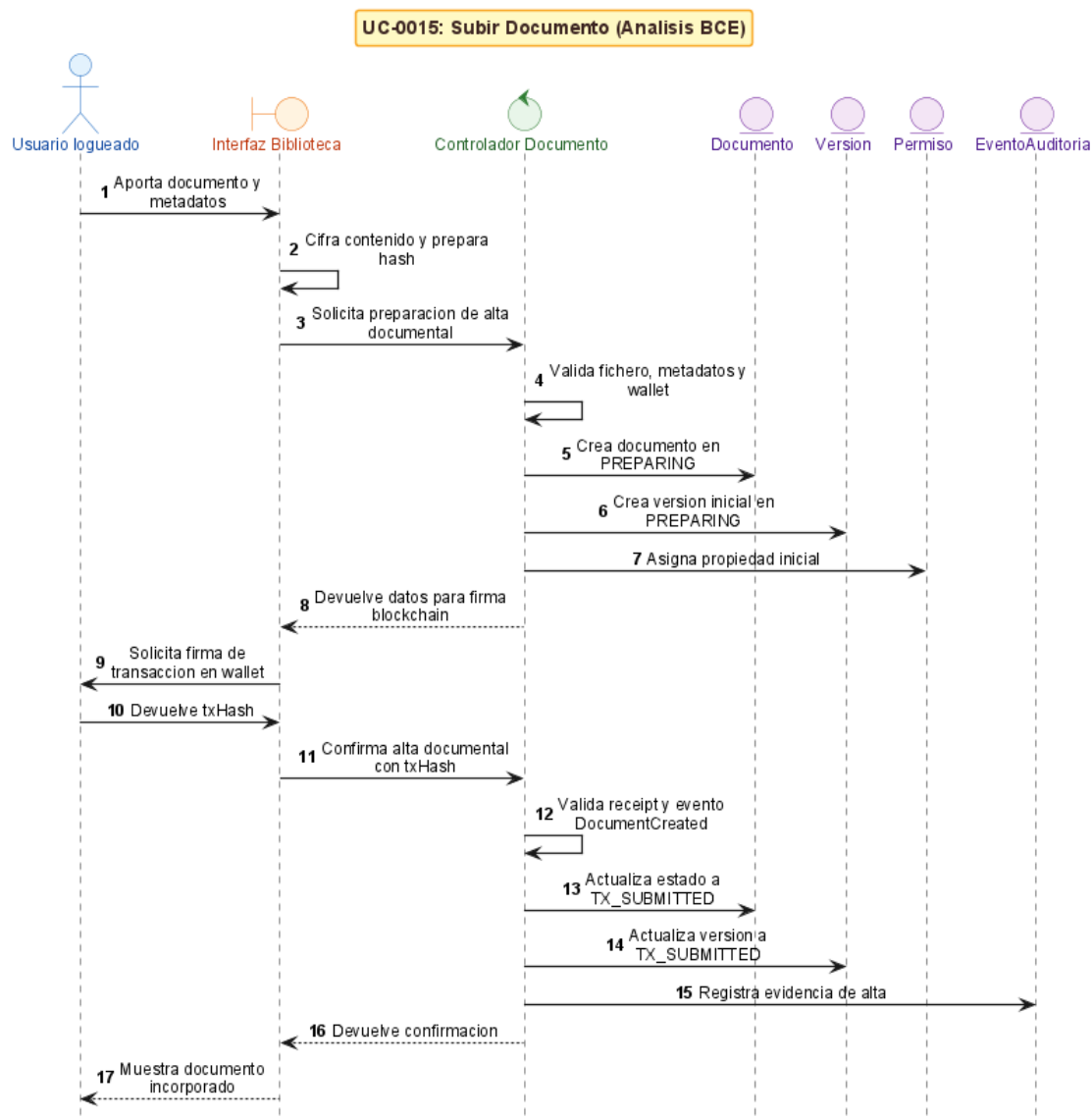


Figura 2: Diagrama de secuencia de analisis BCE: UC-0015 Subir Documento

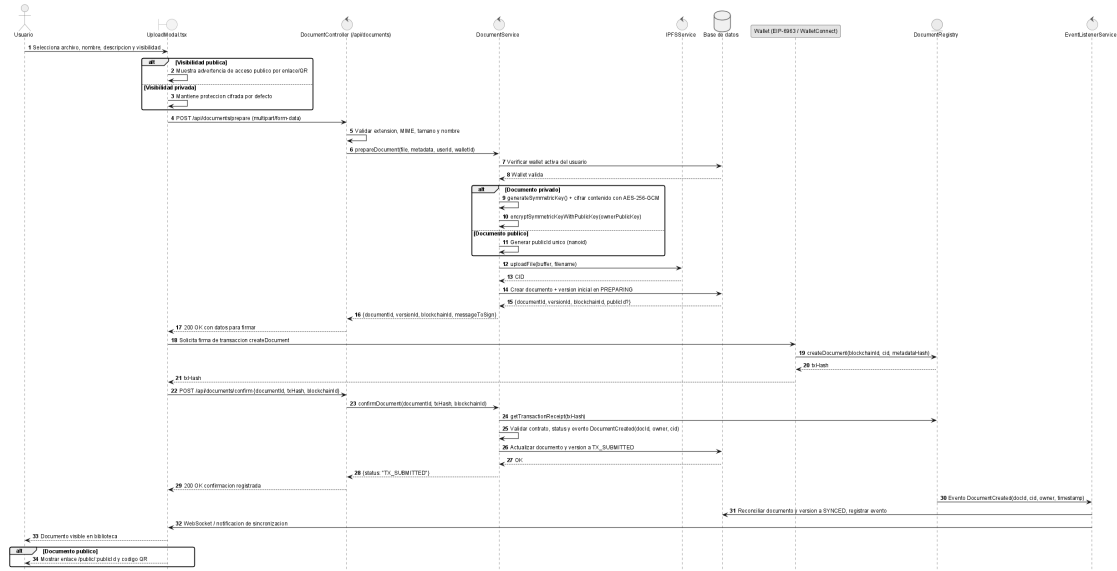


Figura 3: Diagrama de secuencia de diseño: UC-0015 Subir Documento

Para obtener mas informacion sobre este apartado, consultar el Anexo III (Análisis y diseño del sistema).

6.6. Diseño del sistema

Una vez definidos todos los casos de uso y analizados, se procedió a diseñar el sistema, preparando el proyecto para su implementación. La arquitectura de DocumentChain se organiza en capas funcionales desacopladas, con tres subsistemas de persistencia coordinados: PostgreSQL para estado relacional mutable, IPFS para contenido binario direccionable por hash, y Ethereum para registro verificable e inmutable.

Entre las decisiones de diseño mas relevantes destacan: la elección de Ethereum/Hardhat frente a Hyperledger Fabric o Bitcoin por su ecosistema de herramientas y compatibilidad con wallets estandar; la adopción de IPFS con nodo Kubo autoalojado frente al almacenamiento cloud tradicional; y la selección de PostgreSQL con Prisma ORM por su robustez en entornos concurrentes y su integración con TypeScript.

El modelo de permisos on-chain distingue tres roles (VIEWER, EDITOR, OWNER) gestionados directamente por el contrato **DocumentRegistry**, y la seguridad se abordó mediante defensa en profundidad, combinando cifrado AES-256-GCM para documentos privados, protección de claves con criptografía asimétrica y controles de sesión en el backend.

Las Figuras 4 y 5 muestran, respectivamente, el modelo de despliegue con contenedores Docker y el diagrama entidad-relación de la base de datos PostgreSQL. Las Figuras 6 y 7 ilustran los dos patrones arquitectónicos fundamentales del sistema.

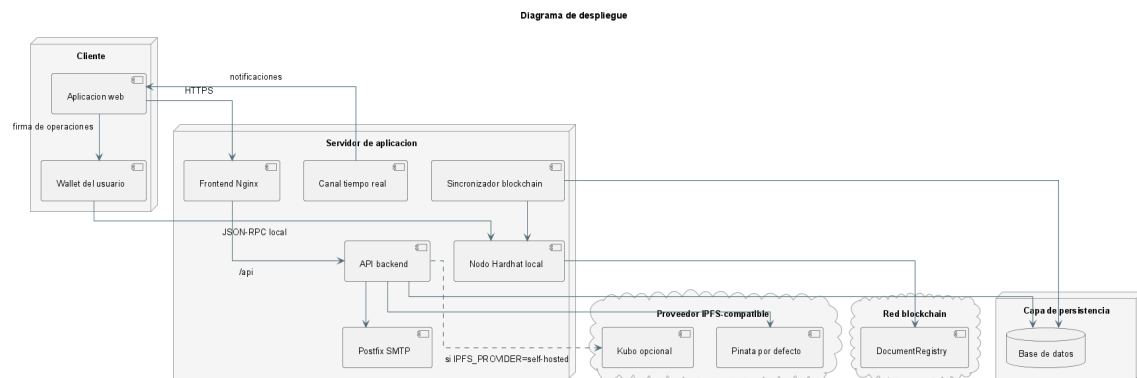


Figura 4: Modelo de despliegue con contenedores Docker

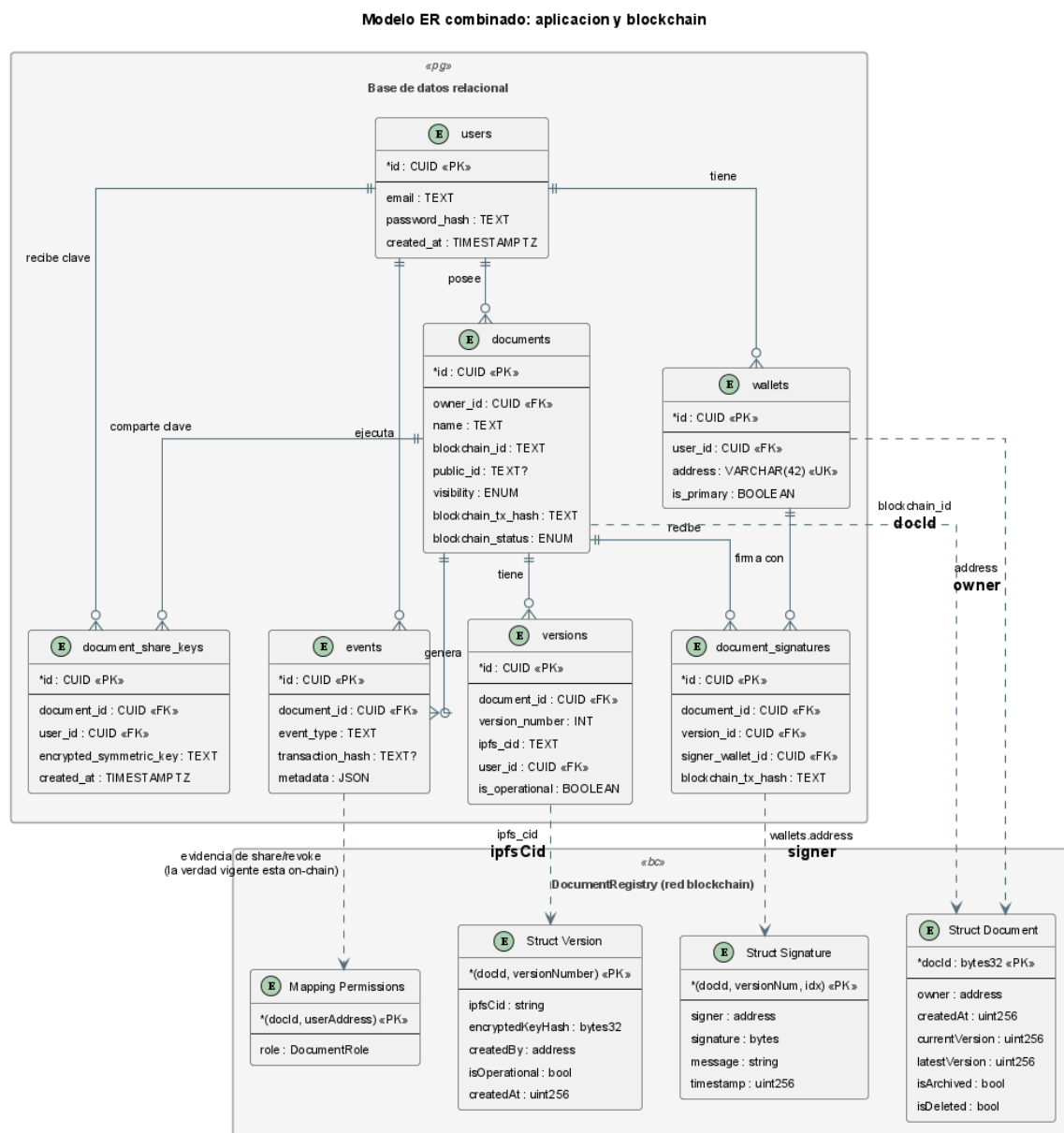


Figura 5: Diagrama entidad-relacion de la base de datos

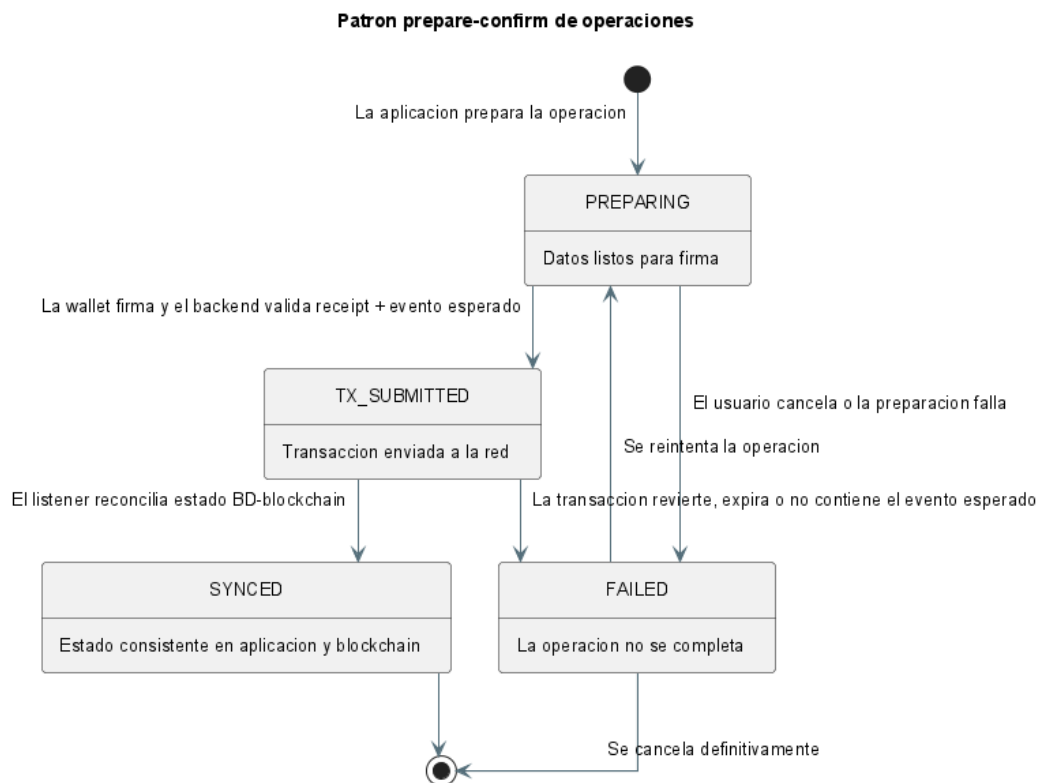


Figura 6: Patron prepare/confirm para transacciones blockchain

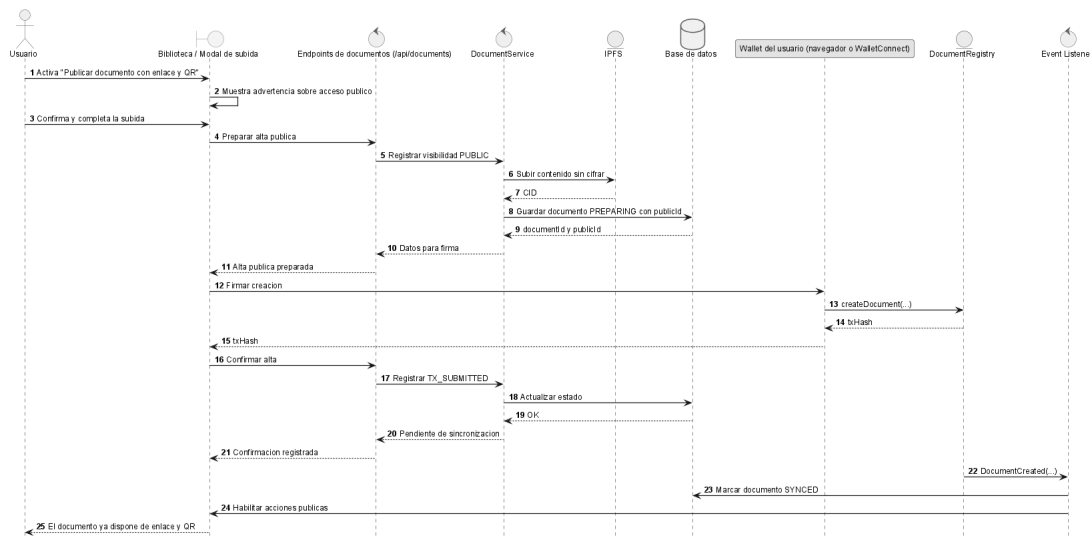


Figura 7: Flujo de sincronizacion bidireccional mediante Event Listener

La Tabla 6 resume los principales riesgos identificados y sus medidas de mitigacion.

ID	Riesgo	Prob.	Impacto	Mitigacion
R-01	Complejidad de integracion blockchain	Alta	Alto	Patron prepare/confirm; listener + worker
R-02	Incompatibilidad entre wallets y EIP-6963	Media	Medio	Soporte multicartera; fallback a WalletConnect
R-03	Fallos en sincronizacion BD-blockchain	Media	Alto	Worker periodico de reconciliacion
R-04	Perdida de contenido IPFS	Baja	Alto	Pin persistente de CIDs; backup periodico
R-05	Vulnerabilidad en smart contract	Baja	Critico	Analisis estatico Slither; patrones OpenZeppelin
R-06	Rendimiento insuficiente bajo carga	Baja	Medio	Limites de recursos; paginacion en todos los listados

Cuadro 6: Registro de riesgos del proyecto

Para obtener mas informacion sobre este apartado, consultar el Anexo III (Analisis y disenio del sistema) y el Anexo V (Plan de seguridad).

6.7. Implementacion

Durante esta fase se ha realizado la codificacion de todo el sistema tomando los resultados obtenidos en la fase de disenio como base. La implementacion se dividio en cuatro secciones: servidor web (frontend React + TypeScript con Vite), servidor API (backend Node.js + Express), contratos inteligentes (Solidity + Hardhat) e infraestructura (Docker Compose).

Las decisiones tecnicas mas relevantes incluyen el disenio de una arquitectura de seguridad por capas (JWT con verificacion de email, AES-256-GCM con intercambio ECDH P-256 y registro blockchain), el flujo prepare/confirm para transacciones blockchain, la integracion de wallets mediante EIP-6963 evitando el vendor lock-in, y la sincronizacion bidireccional blockchain-base de datos mediante EventListenerService y worker de reconciliacion periodica. La contenerizacion con Docker Compose garantiza la reproducibilidad del entorno de ejecucion, orquestando siete servicios.

Para obtener mas informacion sobre este apartado, consultar el Anexo IV (Documentacion tecnica) y el Anexo VII (Manual de montaje).

6.8. Funcionalidad del sistema

En este apartado se muestran los aspectos mas importantes de la funcionalidad del sistema. La aplicacion dispone de un panel principal de gestion documental donde los usuarios pueden realizar todas las operaciones del ciclo de vida completo del documento, una pagina de verificacion publica accesible sin autentificacion que permite verificar la autenticidad de cualquier documento, y un panel de administracion para la gestion de usuarios, roles y supervision del sistema.

6.8.1. Gestion documental

La Figura 8 muestra la vista principal de gestion documental, donde los usuarios pueden listar, buscar, filtrar, subir nuevos documentos y acceder a las operaciones de cada documento (firmar, compartir, versionar, archivar, transferir). La Figura 9 presenta el modal de subida de documentos, con campos de metadatos y seleccion de visibilidad.

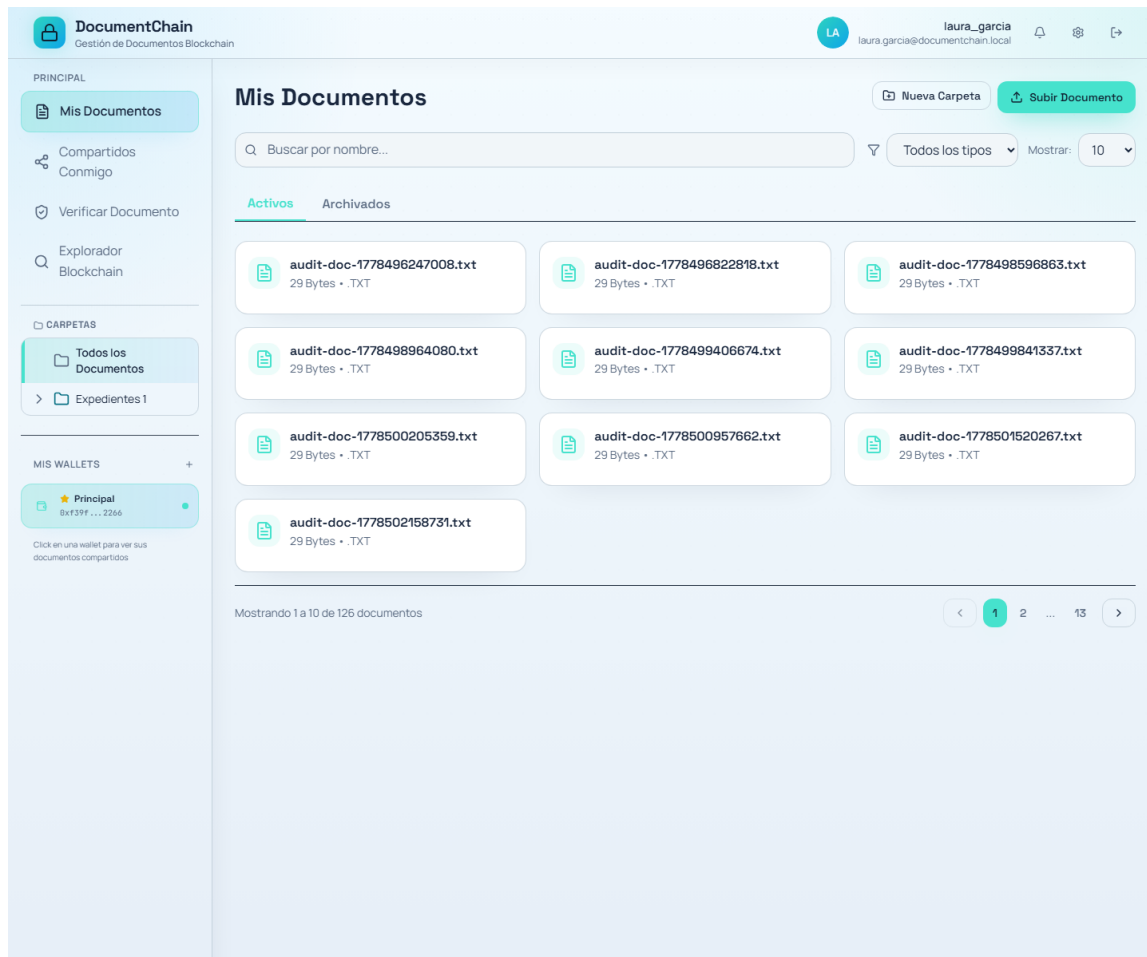


Figura 8: Vista principal de gestion documental

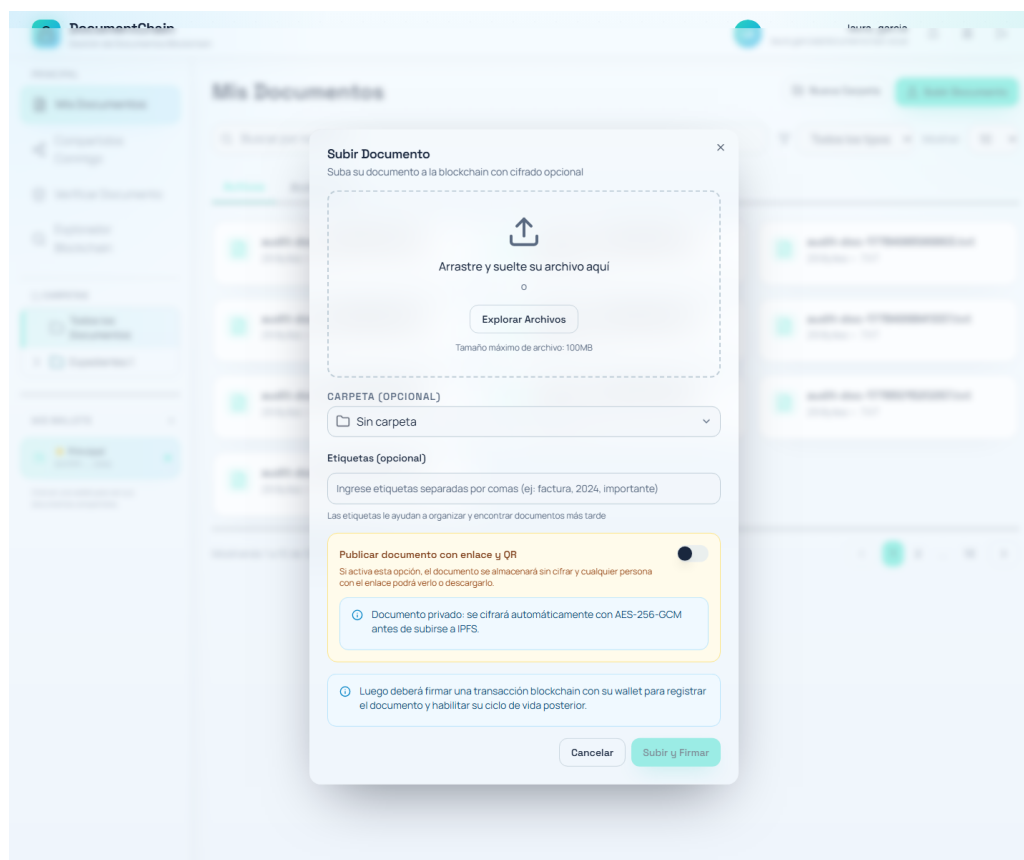


Figura 9: Modal de subida de documentos con metadatos

6.8.2. Firma digital y comparticion

Las Figuras 10 y 11 muestran, respectivamente, el flujo de firma digital de un documento mediante wallet blockchain y el modal de comparticion con configuracion de permisos.

Firmar Documento

Documento

expediente_auditoria_revision_laura_garcia.docx

Versión a firmar

VERSIÓN 1

Comentario (opcional)

Ej: Revisado y aprobado, Conforme con el contenido...

0/200 caracteres

Al firmar, crearás una firma digital inmutable en blockchain. Se te pedirá que firmes con tu wallet conectada.

Cancelar

 Firmar Documento

Figura 10: Modal de firma digital con wallet blockchain

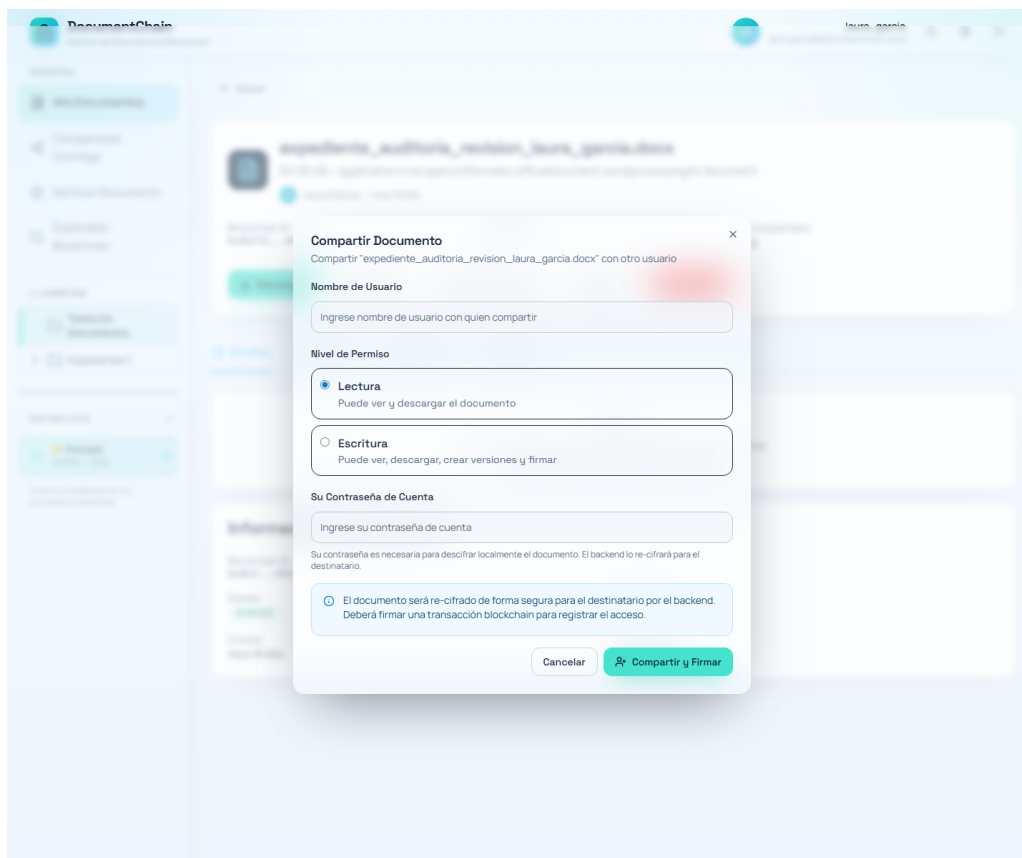


Figura 11: Modal de comparticion de documentos con permisos

6.8.3. Verificacion publica y administracion

La Figura 12 muestra la pagina de verificacion publica, accesible sin necesidad de autentificacion, que permite a cualquier tercero comprobar la autenticidad de un documento mediante su hash o el identificador de la transaccion blockchain. La Figura 13 presenta el panel de administracion para la gestion de usuarios y supervision del sistema.

 **DocumentChain**
Gestión de Documentos Blockchain

Iniciar Sesión Registrarse

Verificar Documento

Verifique la autenticidad del documento usando registros blockchain

Elegir Método de Verificación

**Subir Archivo**
Verificar subiendo el archivo original

**Hash IPFS**
Verificar usando el hash de contenido IPFS

**ID Blockchain**
Verificar usando el ID de documento blockchain

Introducir Datos de Verificación

Seleccionar Archivo a Verificar

Choose File

No file chosen

Verificar Documento

Figura 12: Pagina de verificacion publica de autenticidad de documentos

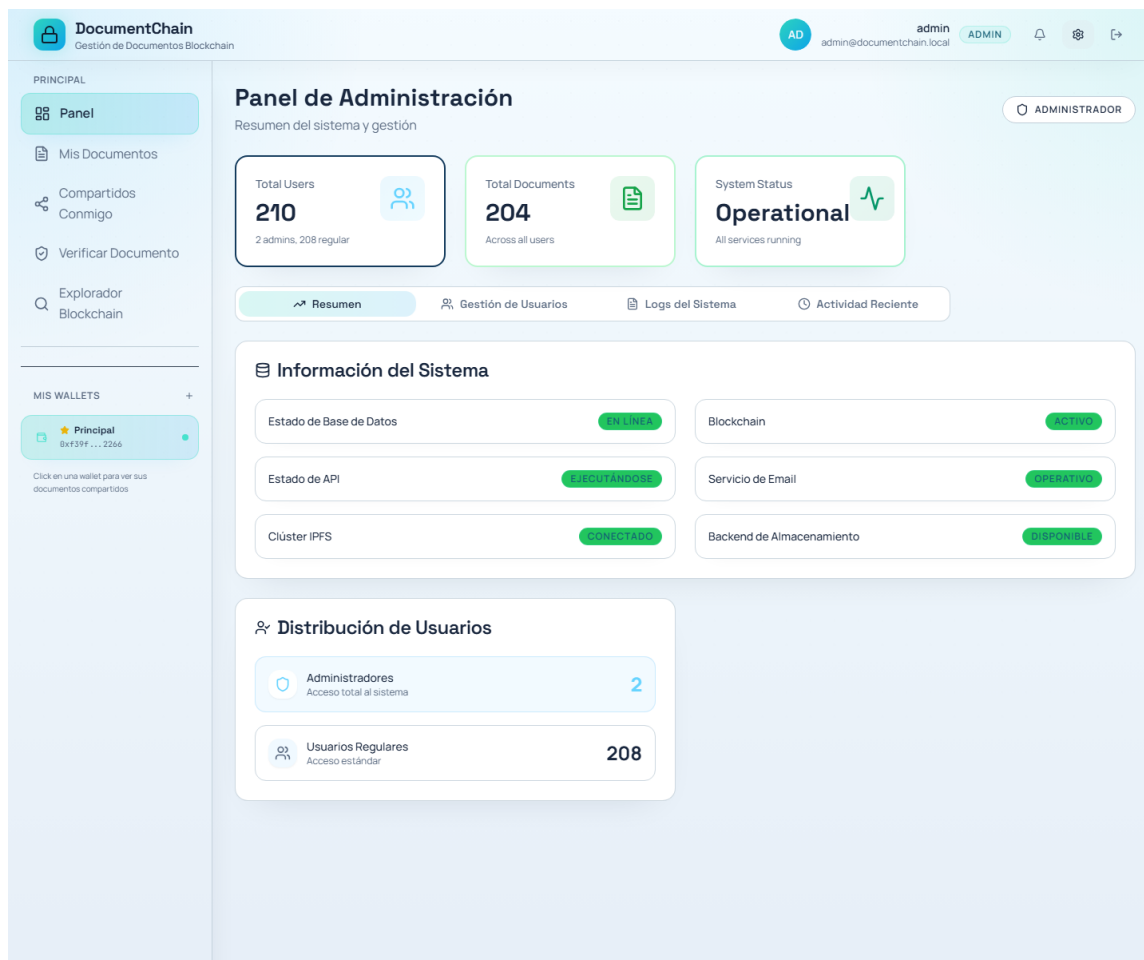


Figura 13: Panel de administracion del sistema

Para obtener mas informacion sobre este apartado, consultar el Anexo VI (Manual de usuario).

7. Limitaciones del sistema

Durante el desarrollo de este proyecto se han encontrado una serie de limitaciones que condicionan su despliegue en entornos de produccion.

La Tabla 7 resume las funcionalidades diferidas, su viabilidad tecnica estimada y la justificacion de su exclusion.

Funcionalidad diferida	Viabilidad	Justificacion de exclusion
Red blockchain publica	Alta	Requiere presupuesto para gas y auditoria de seguridad
Notificaciones push	Alta	Dependiente de app movil; email/in-app cubren el 80 % de casos
App movil nativa	Media	Rediseño UI/UX completo; no port directo
Sistema de pagos	Alta	Proyecto empresarial independiente; no academico
Certificacion eIDAS	Alta	Proceso de >12 meses y coste de auditoria EEC

Cuadro 7: Funcionalidades diferidas por restricciones temporales

7.1. Restricciones tecnologicas y de infraestructura

El sistema se ha desarrollado y probado sobre una red local de Hardhat, no sobre una blockchain real. Esto es suficiente para una demostracion, pero no refleja las condiciones de un entorno de produccion. En una red publica, cada operacion consume gas y los tiempos de confirmacion pueden variar mucho. No se ha medido como afectaria esto a la experiencia de usuario.

Otro punto debil es que el backend actual es un unico proceso. Si el sistema recibiera muchas peticiones simultaneas, probablemente se quedaria corto. Pasar a una arquitectura con varias instancias requeriria repensar como se escuchan los eventos de la blockchain para no procesar el mismo evento dos veces.

Tampoco hay un plan de copias de seguridad. PostgreSQL guarda los datos en un volumen Docker, pero si ese volumen se pierde, se pierden todos los metadatos. La blockchain conservaria los registros, pero sin los metadatos el sistema dejaria de ser util.

7.2. Restricciones de seguridad y auditoria

Aunque se han seguido buenas practicas de seguridad, hay que reconocer varias limitaciones importantes. El contrato inteligente no ha pasado por una auditoria profesional. Se ha usado Slither para analisis estatico y se han seguido las recomendaciones de OpenZeppelin, pero una auditoria externa encontraria casi con seguridad aspectos a mejorar.

Las pruebas de seguridad se han limitado a tests funcionales. No se han realizado pruebas de penetracion ni se han explorado vectores de ataque como front-running, manipulacion de eventos WebSocket o condiciones de carrera en el acceso a la base de datos.

Ademas, la seguridad del sistema depende de que el usuario proteja bien su contrasena y su clave de recuperacion. Si las pierde, pierde el acceso a sus documentos y el sistema no puede hacer nada al respecto, porque ni siquiera el servidor tiene acceso a las claves de cifrado.

7.3. Acceso a usuarios y pruebas de usabilidad

No se ha podido contar con usuarios reales para probar la aplicacion. Las pruebas se han hecho con un grupo muy reducido de personas, todas ellas con perfil tecnico, lo que introduce un sesgo importante. Conceptos como wallet, blockchain o CID, que para un informatico son familiares, pueden resultar confusos para alguien que solo quiere gestionar sus documentos.

Tampoco se han hecho pruebas A/B comparando distintos disenos, ni se ha realizado una auditoria de accesibilidad formal para comprobar que la aplicacion funciona con lectores de pantalla u otras tecnologias asistivas.

7.4. Restricciones regulatorias y de compliance

El RGPD establece el derecho al olvido, pero la blockchain es inmutable por diseño. Si un usuario ejerce este derecho, se puede eliminar su información de la base de datos, pero los hashes y las direcciones Ethereum que quedaron registrados en la cadena no se pueden borrar. Esto crea una tensión que no tiene una solución sencilla.

En cuanto al almacenamiento en IPFS, los archivos se distribuyen por nodos de todo el mundo, lo que podría entrar en conflicto con normativas que exigen que ciertos datos permanezcan dentro de un territorio concreto.

7.5. Restricciones del proyecto

El proyecto se ha desarrollado en unos seis meses por una única persona, con dedicación parcial. Esta limitación de tiempo ha obligado a priorizar. Algunas funcionalidades que habría sido interesante incluir se quedaron fuera del alcance por este motivo, como la migración a una red pública de pruebas, las notificaciones push nativas o un sistema de recuperación de cuenta más amigable para el usuario no técnico.

8. Conclusiones

8.1. Validación de objetivos

Una vez terminado el proyecto, se puede decir que los objetivos que se plantearon al principio se han cumplido. Ha sido un trabajo largo, con momentos complicados, pero el resultado es una plataforma que funciona y que demuestra que se puede hacer gestión documental con garantías blockchain sin renunciar a una experiencia de usuario razonable.

A nivel personal, el proyecto me ha permitido tocar prácticamente todas las capas de una aplicación moderna: frontend con React, backend con Node.js, base de datos con PostgreSQL, almacenamiento descentralizado con IPFS y, por supuesto, blockchain con Solidity. Muchas de estas tecnologías las conocí durante la carrera, pero nunca las había integrado en un mismo sistema. Ha sido un aprendizaje intenso.

También he podido aplicar metodologías de ingeniería del software que había estudiado en asignaturas como Ingeniería del Software (Proceso Unificado, UML) o Gestión de Proyectos (Use Case Points, planificación con Gantt). Ver cómo los diagramas de casos de uso se transforman en endpoints REST y cómo una estimación en horas se convierte en un calendario real ha sido de las partes más satisfactorias del proyecto.

La Tabla 8 resume que tal ha ido cada objetivo.

Objetivo	Descripcion	Estado
OBJ-0001	Gestionar ciclo de vida completo de documentos (crear, versionar, compartir, firmar, archivar, eliminar)	Alcanzado
OBJ-0002	Garantizar autenticidad e integridad mediante registro inmutable en blockchain	Alcanzado
OBJ-0003	Firma digital con wallets blockchain compatibles EIP-6963	Alcanzado
OBJ-0004	Comparticion controlada con permisos on-chain (VIEWER, EDITOR, OWNER)	Alcanzado
OBJ-0005	Auditoria publica de transacciones mediante endpoint por txHash	Alcanzado
OBJ-0006	Confidencialidad con cifrado de extremo a extremo (AES-256-GCM + ECDH)	Alcanzado
OBJ-0007	Gestion de multiples wallets con identificacion mediante EIP-6963	Alcanzado
OBJ-0008	Panel de administracion con gestion de usuarios, roles y supervision del sistema	Alcanzado

Cuadro 8: Validacion de objetivos funcionales

Mas alla de los objetivos concretos, creo que el trabajo aporta algo interesante: una arquitectura documentada que separa lo que necesita ser inmutable (blockchain) de lo que necesita ser rapido (PostgreSQL, IPFS). No es una idea nueva, pero tenerla implementada y explicada paso a paso, con diagramas, manuales y codigo fuente, puede servir como referencia para otros estudiantes o desarrolladores que quieran explorar este camino.

Para mas detalle sobre la planificacion y como se ajusto a lo estimado, se puede consultar el Anexo II.

9. Lineas de trabajo futuras

Ya que ningun diseno es perfecto, se plantean una serie de ideas para mejorar el sistema:

- **Migracion a redes blockchain publicas:** desplegar el contrato DocumentRegistry en una red de pruebas publica y, tras auditoria de seguridad profesional, migrar a una red principal o Layer 2 con un sistema de meta-transacciones para eliminar la barrera del gas para el usuario final.
- **Aplicacion movil nativa o PWA:** desarrollar una aplicacion para iOS/Android o convertir el frontend actual en una Progressive Web App instalable, con almacenamiento seguro de claves y notificaciones push nativas.
- **Mejoras de usabilidad:** implementar un tour interactivo guiado para nuevos usuarios, modelo de cuenta abstracta (ERC-4337) para abstraer la complejidad de las wallets, y visores de documentos integrados en la interfaz.
- **Funcionalidades avanzadas:** firma colectiva multisig con umbrales de firmantes, plantillas de documentos personalizables, flujos de aprobacion configurables y busqueda semantica sobre el contenido textual de los documentos.

- **Sostenibilidad del proyecto:** publicar el código bajo licencia open source (MIT), mantener documentación OpenAPI actualizada y establecer un programa de recompensas por bugs.

10. Referencias y bibliografía

1. Karner, G. (1993). "Resource Estimation for Objectory Projects". Objective Systems SF AB.
2. Nakamoto, S. (2008). "Bitcoin: A Peer-to-Peer Electronic Cash System".
3. Wood, G. (2014). "Ethereum: A Secure Decentralised Generalised Transaction Ledger". Ethereum Project Yellow Paper.
4. Benet, J. (2014). "IPFS - Content Addressed, Versioned, P2P File System". arXiv:1407.3561.
5. Loshin, P. (2013). "IPv6 Addressing and Subnetting". John Wiley & Sons.
6. INCIBE (2023). "Plan Director de Seguridad". Instituto Nacional de Ciberseguridad.
7. OWASP Foundation (2021). "OWASP Top 10:2021".
8. React Documentation (2025). "React 19 Documentation". Meta Platforms.
9. Prisma Documentation (2025). "Prisma ORM Documentation". Prisma Data.
10. Hardhat Documentation (2025). "Hardhat Documentation". Nomic Foundation.
11. IEEE (2019). "IEEE Reference Guide". IEEE Author Center.
12. Union Europea (2014). "Reglamento (UE) n.º 910/2014 del Parlamento Europeo y del Consejo, de 23 de julio de 2014, relativo a la identificación electrónica y los servicios de confianza para las transacciones electrónicas en el mercado interior (eIDAS)". Diario Oficial de la Unión Europea, L 257/73.
13. Union Europea (2016). "Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD)". Diario Oficial de la Unión Europea, L 119/1.
14. ACFE (2024). "Occupational Fraud 2024: A Report to the Nations". Association of Certified Fraud Examiners.
15. Buterin, V. (2014). "Ethereum: Platform Review". Opportunities and Challenges for Private and Consortium Blockchains.
16. Antonopoulos, A. M., & Wood, G. (2018). "Mastering Ethereum: Building Smart Contracts and DApps". O'Reilly Media.
17. Wohrer, M., & Zdun, U. (2018). "Smart Contracts: Security Patterns in the Ethereum Ecosystem and Solidity". International Workshop on Blockchain Oriented Software Engineering (IWBOSE).
18. Merkle, R. C. (1980). "Protocols for Public Key Cryptosystems". IEEE Symposium on Security and Privacy.

19. Daemen, J., & Rijmen, V. (2002). "The Design of Rijndael: AES - The Advanced Encryption Standard". Springer.
20. Swartz, A. (2014). "Bitcoin: Its History, Technology and Governance". *Communications of the ACM*, 57(7).
21. OpenZeppelin (2023). "OpenZeppelin Contracts Documentation v5.0". OpenZeppelin Community.
22. Trail of Bits (2023). "Smart Contract Security Field Guide". Trail of Bits, Inc.
23. Docker Inc. (2024). "Docker Documentation: Compose File Specification". Docker, Inc.
24. Mozilla Developer Network (2024). "Web Crypto API Documentation". MDN Web Docs.
25. Zyskind, G., Nathan, O., & Pentland, A. (2015). "Decentralizing Privacy: Using Blockchain to Protect Personal Data". *IEEE Security and Privacy Workshops*.
26. Liang, X., Shetty, S., Tosh, D., Kamhoua, C., Kwiat, K., & Njilla, L. (2017). "ProvChain: A Blockchain-based Data Provenance Architecture in Cloud Environment with Enhanced Privacy and Availability". *IEEE/ACM CCGrid*.
27. Ateniese, G., Magri, B., Venturi, D., & Andrade, E. (2017). "Redactable Blockchain – or – Rewriting History in Bitcoin and Friends". *IEEE European Symposium on Security and Privacy*.
28. Brooke, J. (1996). "SUS: A Quick and Dirty Usability Scale". *Usability Evaluation in Industry*. Taylor & Francis.
29. Bangor, A., Kortum, P., & Miller, J. (2009). "Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale". *Journal of Usability Studies*, 4(3).